SSH BRUTE FORCE ATTACK

Simulation, Log Analysis & Detection Using Splunk

Security Information and Event Management

Failed-Login Detection on a Kali Linux SSH Server

OpenSSH • auth.log • Splunk Enterprise

Prepared by Anjali · October 2026

1. Executive Summary

This report documents a controlled SSH brute-force simulation and its detection in a virtualized home lab. A Kali Linux virtual machine running on VMware Workstation acted as the SSH server. Failed logins were generated against it using a deliberately invalid username, `wronguser`, and the evidence was then followed through the whole detection chain: the OpenSSH service, the `/var/log/auth.log` file, command-line analysis with `grep` and `tail`, a Splunk Enterprise search, and a real-time Splunk alert named `ssh brute force`.

Each step is explained alongside the screenshot that proves it. The report also records the errors met along the way (an SSH service that was not running, a log file path that did not exist, and a search phrase that matched the wrong lines) together with the cause and the fix for each one.

Key Achievements

- Started the OpenSSH server and confirmed it was listening on port 22 (IPv4 and IPv6)
- Identified the server address, `192.168.186.133`, with `ip a`
- Generated repeated failed SSH logins using an invalid username
- Located and filtered the evidence in `/var/log/auth.log` with `grep` and `tail`
- Interpreted each sshd and PAM message (Invalid user, Failed password, authentication failure)
- Confirmed the same events in Splunk (`host=kali`, `sourcetype=linux:auth`)
- Verified that the real-time `ssh brute force` alert fired (15 triggered alerts, Medium severity)
- Diagnosed and fixed three errors, documented in Section 6

2. Introduction

2.1 What is SSH?

Secure Shell (SSH) is the standard protocol for secure remote administration of Linux systems. An SSH server (`sshd`) listens on TCP port 22, authenticates each user by password or public key, and only then grants a shell. Because it is exposed to the network and protects administrative access, it is one of the most attacked services on the internet.

2.2 What is an SSH Brute-Force Attack?

In a brute-force (password-guessing) attack, an attacker repeatedly tries usernames and passwords until one works. Each wrong guess leaves a trace in the server log, so the attack is noisy and detectable: many `Failed password` messages, often for usernames that do not exist (`Invalid user`), from the same source within a short time.

2.3 Why Monitor SSH with a SIEM?

A SIEM collects logs from many hosts into one searchable place and can raise alerts automatically. Instead of reading `auth.log` by hand on every server, an analyst writes a search once and lets the SIEM watch continuously. This lab shows that principle at small scale with Splunk Enterprise.

2.4 Tools Used

- **OpenSSH server (sshd)**: the service being tested, controlled with `systemctl`
- **ip, ssh, grep, tail**: standard Linux tools for addressing, login testing and log analysis
- **/var/log/auth.log**: the Debian/Kali authentication log that records sshd, PAM and sudo activity

- **Splunk Enterprise**: SIEM used to search the log events and raise the real-time alert

> **LAB BOUNDARY**
> All testing was performed against the author's own virtual machine in an isolated lab. Never run login-guessing tests against systems you do not own or do not have written permission to test.

3. Lab Environment & Architecture

3.1 Virtual Machine Configuration

Component	Detail
Hypervisor	VMware Workstation (Windows host)
Guest operating system	Kali Linux (listed as "Debian 12.x 64-bit" in the VMware library)
Role	SSH server (target), test client and monitored endpoint
Hostname / user	`kali` / `kali` (uid 1000)
IP address	`192.168.186.133/24` on `eth0` (dynamic, from DHCP)
SSH server	OpenSSH `sshd`, TCP port 22
SIEM	Splunk Enterprise, Search & Reporting app
Log source	`/var/log/auth.log` (Splunk shows `host=kali`, `sourcetype=linux:auth`)

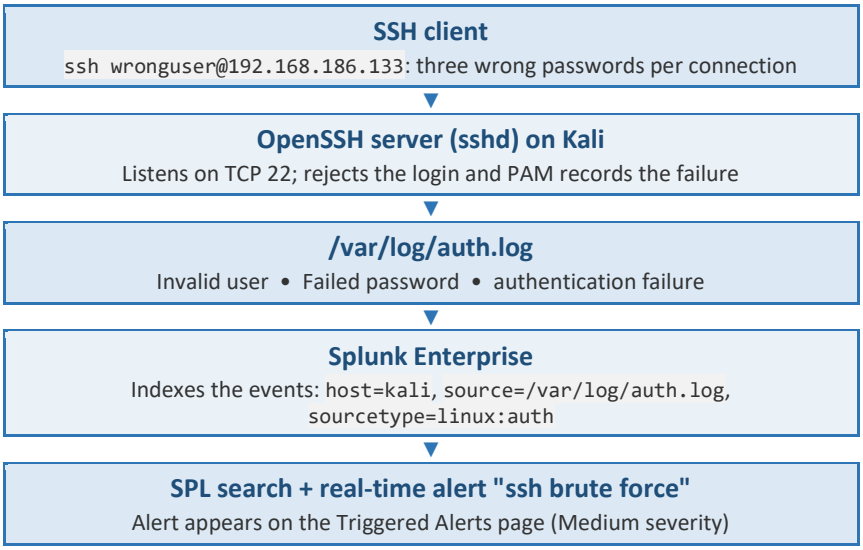
3.2 Network Architecture

A single VM plays both roles in this lab. The `ssh` client on Kali connects to Kali's own address, `192.168.186.133`, so the log records that same address as the source of the failed logins. This keeps the lab simple and fully contained.

> **REAL-WORLD DIFFERENCE**
> In a genuine attack the source address in `auth.log` would belong to a remote machine. Everything else (the log messages, the Splunk search and the alert) works the same way.

3.3 Detection Data Flow

The diagram below shows how a single failed login travels from the SSH client to a Splunk alert.



4. Implementation: Step-by-Step

This section follows the lab in the order it was performed. Every step gives the purpose, the command, the screenshot, and an explanation of what the output means.

4.1 Step 1: Checking and Starting the SSH Service

The OpenSSH server must be running before any login can be tested. If it is stopped, nothing listens on TCP port 22, so there is nothing to receive or log authentication attempts. On Kali (a systemd-based system) the service is queried and controlled with `systemctl`.

Command used:
```
sudo systemctl status ssh
sudo systemctl start ssh
sudo systemctl status ssh
```

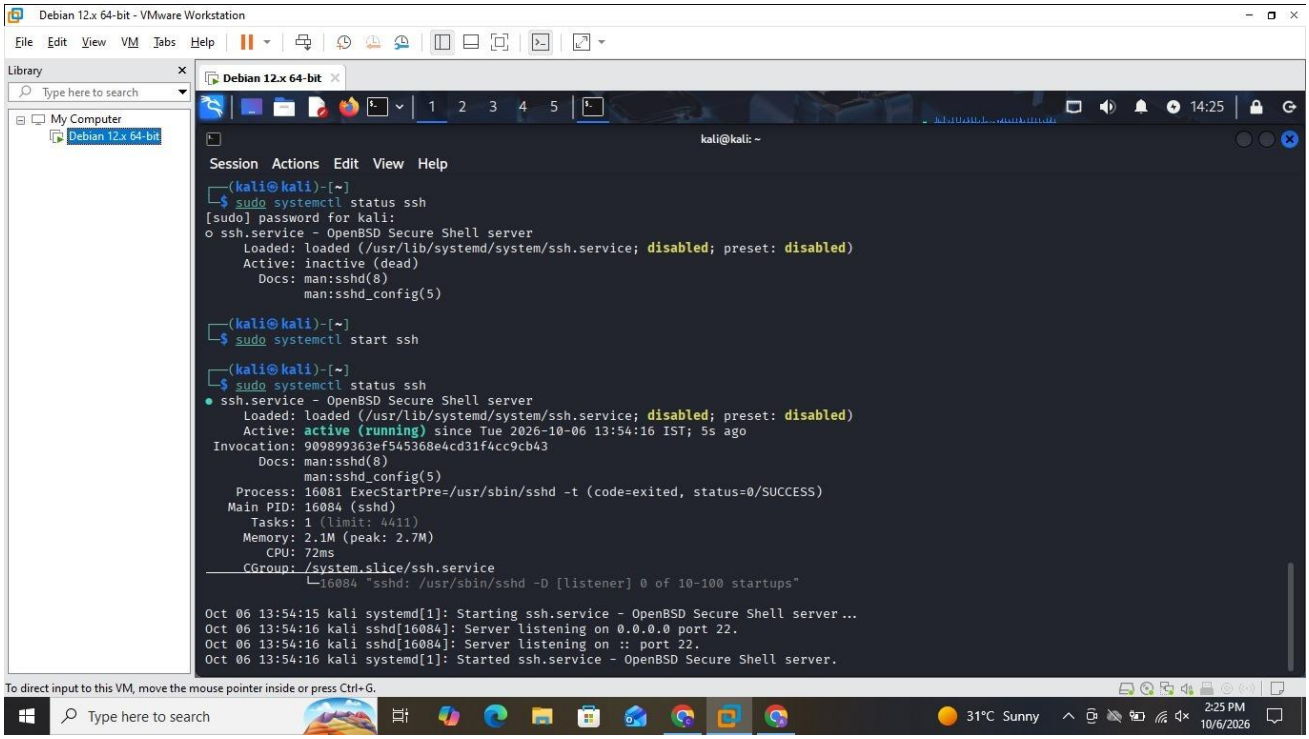


Figure 1. *SSH service shown inactive, started with systemctl, then confirmed active (running).*

What the output shows

- **First status:** `Loaded: ... disabled; preset: disabled` and `Active: inactive (dead)` mean the service is installed but not running, and is not set to start at boot. The hollow circle (○) in front of `ssh.service` also marks it as stopped.
- **`sudo systemctl start ssh`** prints nothing when it succeeds.
- **Second status:** `Active: active (running) since Tue 2026-10-06 13:54:16 IST` with the main process `sshd` (PID 16084), shown with a green dot.
- **`ExecStartPre=/usr/sbin/sshd -t (status=0/SUCCESS)`** shows the configuration file passed its syntax check before the service launched.
- **Journal lines:** `Server listening on 0.0.0.0 port 22` (all IPv4 interfaces) and `Server listening on :: port 22` (all IPv6 interfaces).
- **[listener] 0 of 10-100 startups:** no unauthenticated connections were waiting at that moment.

ERROR ENCOUNTERED
The service was initially `inactive (dead)`. Cause and fix are explained in Section 6.1.

VERIFICATION RESULT
OpenSSH is active (running) and listening on port 22.

4.2 Step 2: Identifying the SSH Server's IP Address

The login test needs a destination, so the server's address is read from its network interfaces.

Command used: `ip a`

```
┌──(kali㉿kali)-[~]
└─$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:c5:43:58 brd ff:ff:ff:ff:ff:ff
    inet 192.168.186.133/24 brd 192.168.186.255 scope global dynamic noprefixroute eth0
       valid_lft 1077sec preferred_lft 1077sec
    inet6 fe80::20c:29ff:fec5:4358/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
```

Figure 2. *Network interfaces on Kali: eth0 holds 192.168.186.133/24.*

What the output shows

- **`lo`** is the loopback interface (`127.0.0.1`). It only carries traffic from the machine to itself.
- **`eth0`** is the real network adapter. `state UP` means it is connected.
- **`inet 192.168.186.133/24`** is the IPv4 address. `/24` means the network is `192.168.186.0` with broadcast address `192.168.186.255`.
- **`dynamic`** and `valid_lft 1077sec` show the address was leased by DHCP and the lease has about 18 minutes left on this screenshot, so the address can change after a lease renewal or reboot.
- **`link/ether 00:0c:29:c5:43:58`** is the virtual MAC address. The `00:0c:29` prefix belongs to VMware virtual adapters.
- **`inet6 fe80::20c:29ff:fec5:4358/64`** is the IPv6 link-local address.

TIP
Because the address is dynamic, run `ip a` again at the start of each lab session before using it as the SSH destination.

4.3 Step 3: Simulating Failed SSH Logins

To produce failed logins safely, the SSH client was pointed at the server using a username that does not exist on the system, `wronguser`, and wrong passwords were typed on purpose. Using an invalid user makes the server write the very clear `Invalid user` message in its log.

Command used: `ssh wronguser@192.168.186.133`

```
┌──(kali㉿kali)-[~]
└─$ ssh wronguser@192.168.186.133
The authenticity of host '192.168.186.133 (192.168.186.133)' can't be established.
ED25519 key fingerprint is SHA256:1PQd79GHorhA0aY4C3Sl2kQ95HjkXOy81qdyWo/XU/k.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.186.133' (ED25519) to the list of known hosts.
wronguser@192.168.186.133's password:
Permission denied, please try again.
wronguser@192.168.186.133's password:
Permission denied, please try again.
wronguser@192.168.186.133's password:
wronguser@192.168.186.133: Permission denied (publickey,password).
```

Figure 3. *First SSH attempt: host key prompt, then three rejected passwords.*

What happened, line by line

1. **Host key prompt.** On a first connection the client does not know the server. It shows the ED25519 fingerprint and asks `Are you sure you want to continue connecting?`. Answering `yes` stores the key in `~/.ssh/known_hosts` (`Warning: Permanently added ... to the list of known hosts`).

2. **Password prompts.** `wronguser@192.168.186.133's password:` appears three times. Each wrong password gives `Permission denied, please try again.`
3. **Final rejection.** `Permission denied (publickey,password)` ends the attempt. It lists the two methods the server offered (public key and password); both failed, and the client stops after three password prompts, which is its default.

WHY THE SERVER ASKS FOR A PASSWORD FOR A USER THAT DOES NOT EXIST
OpenSSH deliberately behaves the same for real and non-existent users, so the person at the keyboard cannot tell which usernames are valid. The truth is only visible in the server log, which is exactly what the next steps examine.

SCOPE OF THE SIMULATION
This test sends three manual attempts per connection and was repeated a few times. Automated attack tools send many attempts per minute, but they leave the same kind of log messages, so the same detection approach applies.

4.4 Step 4: First Attempts to Search the Log (Errors Encountered)

On Debian-based systems such as Kali, sshd reports through the system logger into `/var/log/auth.log`. The first search used a file name that does not exist on this system.

Command used: `sudo grep "Failed Attempts" /var/log/pract_auth.log`

```
┌──(kali㉿kali)-[~]
└─$ sudo grep "Failed Attempts" /var/log/pract_auth.log
grep: /var/log/pract_auth.log: No such file or directory
```

Figure 4. *grep reports "No such file or directory" for the wrong file name.*

`grep` answered `No such file or directory`. The file name was corrected to `/var/log/auth.log`, but the search phrase was still wrong:

Command used: `sudo grep "Failed Attempts" /var/log/auth.log`

```
┌──(kali㉿kali)-[~]
└─$ sudo grep "Failed Attempts" /var/log/auth.log
2026-10-06T13:57:12.173235+05:30 kali sudo:     kali : TTY=pts/0 ; PWD=/home/kali ; USER=root ; COMMAND=/usr/bin/grep 'Failed Attempts' /var/
log/pract_auth.log
2026-10-06T13:57:23.414529+05:30 kali sudo:     kali : TTY=pts/0 ; PWD=/home/kali ; USER=root ; COMMAND=/usr/bin/grep 'Failed Attempts' /var/
log/auth.log
```

Figure 5. *The only matches are sudo's own records of the earlier grep commands.*

This time grep returned two lines at 13:57:12 and 13:57:23, but neither is an SSH event. Both are `sudo` entries that record the grep commands themselves (`COMMAND=/usr/bin/grep 'Failed Attempts' ...`). OpenSSH never writes the words "Failed Attempts", so the search matched only its own trace.

ERRORS ENCOUNTERED
Wrong file name (Section 6.2) and a search phrase that does not exist in SSH logs (Section 6.3).

4.5 Step 5: Filtering the Correct SSH Failure Messages

To find the real messages, the search phrase was changed to the exact text that sshd writes, and `-E` (extended regular expressions) was used so that `|` means OR.

Command used: `sudo grep -E "Failed password|Invalid user" /var/log/auth.log`

```
┌──(kali㉿kali)-[~]
└─$ sudo grep -E "Failed password|Invalid user" /var/log/auth.log
2026-10-06T13:55:23.685559+05:30 kali sshd-session[16516]: Invalid user wronguser from 192.168.186.133 port 49380
2026-10-06T13:55:29.378038+05:30 kali sshd-session[16516]: Failed password for invalid user wronguser from 192.168.186.133 port 49380 ssh2
2026-10-06T13:55:34.708149+05:30 kali sshd-session[16516]: Failed password for invalid user wronguser from 192.168.186.133 port 49380 ssh2
2026-10-06T13:55:40.759340+05:30 kali sshd-session[16516]: Failed password for invalid user wronguser from 192.168.186.133 port 49380 ssh2
2026-10-06T14:05:25.690232+05:30 kali sudo:     kali : TTY=pts/0 ; PWD=/home/kali ; USER=root ; COMMAND=/usr/bin/grep -E 'Failed password|Inv
alid user' /var/log/auth.log
```

Figure 6. *Four sshd events from the first connection, plus one sudo line.*

What the output shows

- `13:55:23 Invalid user wronguser from 192.168.186.133 port 49380`: the username was rejected as non-existent.
- `13:55:29`, `13:55:34`, `13:55:40 Failed password for invalid user wronguser ... ssh2`: one line per wrong password, roughly 5 to 6 seconds apart (the time taken to type each guess).
- All four share `sshd-session[16516]` and port `49380`, which identifies them as one SSH connection.
- The fifth line (`14:05:25 ... sudo`) is sudo's record of this very grep command. It is noise, explained in Section 6.4.

4.6 Step 6: Repeating the Login Attempts

A brute-force pattern needs repetition, so the failed login was repeated.

Command used: `ssh wronguser@192.168.186.133`

```
┌──(kali㉿kali)-[~]
└─$ ssh wronguser@192.168.186.133
wronguser@192.168.186.133's password:
Permission denied, please try again.
wronguser@192.168.186.133's password:
Permission denied, please try again.
wronguser@192.168.186.133's password:
wronguser@192.168.186.133: Permission denied (publickey,password).
```

Figure 7. *Second round: no host key prompt this time, same three rejections.*

The fingerprint question no longer appears because the server's key is already stored in `known_hosts`. The result is the same three `Permission denied` responses, which create a second block of events in the log.

4.7 Step 7: Viewing Only the Newest Matches

As `auth.log` grows, the filtered output becomes long. Piping it into `tail -20` keeps only the last 20 matching lines.

Command used: `sudo grep -E "Failed password|Invalid user" /var/log/auth.log | tail -20`

```
2026-10-06T13:55:40.759340+05:30 kali sshd-session[16516]: Failed password for invalid user wronguser from 192.168.186.133 port 49380 ssh2
2026-10-06T14:05:25.690232+05:30 kali sudo:     kali : TTY=pts/0 ; PWD=/home/kali ; USER=root ; COMMAND=/usr/bin/grep -E 'Failed password|Inv
alid user' /var/log/auth.log
2026-10-06T14:06:18.890584+05:30 kali sshd-session[22092]: Invalid user wronguser from 192.168.186.133 port 35134
2026-10-06T14:06:23.863504+05:30 kali sshd-session[22092]: Failed password for invalid user wronguser from 192.168.186.133 port 35134 ssh2
2026-10-06T14:06:32.117402+05:30 kali sshd-session[22092]: Failed password for invalid user wronguser from 192.168.186.133 port 35134 ssh2
2026-10-06T14:06:40.334927+05:30 kali sshd-session[22092]: Failed password for invalid user wronguser from 192.168.186.133 port 35134 ssh2
2026-10-06T14:07:08.686437+05:30 kali sudo:     kali : TTY=pts/0 ; PWD=/home/kali ; USER=root ; COMMAND=/usr/bin/grep -E 'Failed password|Inv
alid user' /var/log/auth.log
```

Figure 8. *Newest matches: the second connection (PID 22092, port 35134) appears after the first.*

The new events come from `sshd-session[22092]` on port `35134`: `Invalid user` at 14:06:18, then `Failed password` at 14:06:23, 14:06:32 and 14:06:40. A different process ID and source port compared with the first block (16516 and 49380) prove that this is a new connection, not a continuation of the old one.

4.8 Step 8: Reading the Full Authentication Log

The filter hides the supporting messages. `tail` shows the last 20 lines of the whole file, including the PAM (authentication framework) lines that sit between the sshd lines.

Command used: `sudo tail -20 /var/log/auth.log`

```
┌──(kali㉿kali)-[~]
└─$ sudo tail -20 /var/log/auth.log
2026-10-06T13:57:23.417269+05:30 kali sudo: pam_unix(sudo:session): session opened for user root(uid=0) by kali(uid=1000)
2026-10-06T13:57:23.422125+05:30 kali sudo: pam_unix(sudo:session): session closed for user root
2026-10-06T14:05:25.690232+05:30 kali sudo:     kali : TTY=pts/0 ; PWD=/home/kali ; USER=root ; COMMAND=/usr/bin/grep -E 'Failed password|Inv
alid user' /var/log/auth.log
2026-10-06T14:05:25.694829+05:30 kali sudo: pam_unix(sudo:session): session opened for user root(uid=0) by kali(uid=1000)
2026-10-06T14:05:25.700597+05:30 kali sudo: pam_unix(sudo:session): session closed for user root
2026-10-06T14:06:18.890584+05:30 kali sshd-session[22092]: Invalid user wronguser from 192.168.186.133 port 35134
2026-10-06T14:06:22.015908+05:30 kali sshd-session[22092]: pam_unix(sshd:auth): check pass; user unknown
2026-10-06T14:06:22.016333+05:30 kali sshd-session[22092]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser=
rhost=192.168.186.133
2026-10-06T14:06:23.863504+05:30 kali sshd-session[22092]: Failed password for invalid user wronguser from 192.168.186.133 port 35134 ssh2
2026-10-06T14:06:30.058383+05:30 kali sshd-session[22092]: pam_unix(sshd:auth): check pass; user unknown
2026-10-06T14:06:32.117402+05:30 kali sshd-session[22092]: Failed password for invalid user wronguser from 192.168.186.133 port 35134 ssh2
2026-10-06T14:06:38.368188+05:30 kali sshd-session[22092]: pam_unix(sshd:auth): check pass; user unknown
2026-10-06T14:06:40.334927+05:30 kali sshd-session[22092]: Failed password for invalid user wronguser from 192.168.186.133 port 35134 ssh2
2026-10-06T14:06:42.057711+05:30 kali sshd-session[22092]: Connection closed by invalid user wronguser 192.168.186.133 port 35134 [preauth]
2026-10-06T14:06:42.058240+05:30 kali sshd-session[22092]: PAM 2 more authentication failures; logname= uid=0 euid=0 tty=ssh ruser= rhost=192
.168.186.133
2026-10-06T14:07:08.686437+05:30 kali sudo:     kali : TTY=pts/0 ; PWD=/home/kali ; USER=root ; COMMAND=/usr/bin/grep -E 'Failed password|Inv
alid user' /var/log/auth.log
```

Figure 9. *Last 20 lines of auth.log, showing the complete record of the second connection.*

Message-by-message interpretation

Observed log message	Meaning
`sshd-session[22092]: Invalid user wronguser from 192.168.186.133 port 35134`	The client asked for a username that does not exist. The source address and port are recorded.
`pam_unix(sshd:auth): check pass; user unknown`	PAM tried to check a password for a user it could not find.
`pam_unix(sshd:auth): authentication failure; ... rhost=192.168.186.133`	PAM confirms the failed authentication and records the remote host.
`Failed password for invalid user wronguser from 192.168.186.133 port 35134 ssh2`	One wrong password. `ssh2` is the protocol version. This is the key line for detection and appears three times (14:06:23, 14:06:32, 14:06:40).
`Connection closed by invalid user wronguser ... [preauth]`	The client gave up after three tries. `[preauth]` means the connection ended before authentication succeeded.
`PAM 2 more authentication failures; ...`	PAM summarizes the remaining failures of the same connection (one was logged separately, plus two more, which equals the three wrong passwords).
`sudo: ... session opened for user root(uid=0) by kali(uid=1000)`	The `kali` user ran a command with sudo (here, reading the log). This is not SSH activity.

> **READING THE TIMESTAMPS**
> The `+05:30` at the end of each timestamp is India Standard Time. On current OpenSSH releases each connection is handled by an `sshd-session` process, which is why the log names `sshd-session[PID]` and not `sshd[PID]`.

4.9 Step 9: Generating Another Round for the SIEM

A further set of failed logins was generated so that Splunk and its alert would have fresh events to work on.

Command used: `ssh wronguser@192.168.186.133`

```
┌──(kali㉿kali)-[~]
└─$ ssh wronguser@192.168.186.133
wronguser@192.168.186.133's password:
Permission denied, please try again.
wronguser@192.168.186.133's password:
Permission denied, please try again.
wronguser@192.168.186.133's password:
wronguser@192.168.186.133: Permission denied (publickey,password).

┌──(kali㉿kali)-[~]
└─$
```

Figure 10. *Third round of three failed attempts, ending back at the shell prompt.*

Again the client shows three `Permission denied, please try again.` responses and the final `Permission denied (publickey,password).` Each round adds one `Invalid user`, three `Failed password` and the PAM lines to `auth.log`.

5. Detection in Splunk Enterprise

With the evidence confirmed locally, the same events were checked in Splunk, which is where an analyst would normally work.

5.1 Searching for Failed Logins

In the Search & Reporting app the following search was run with the time range set to **Last 15 minutes**:

```
index=* "Failed password"
```

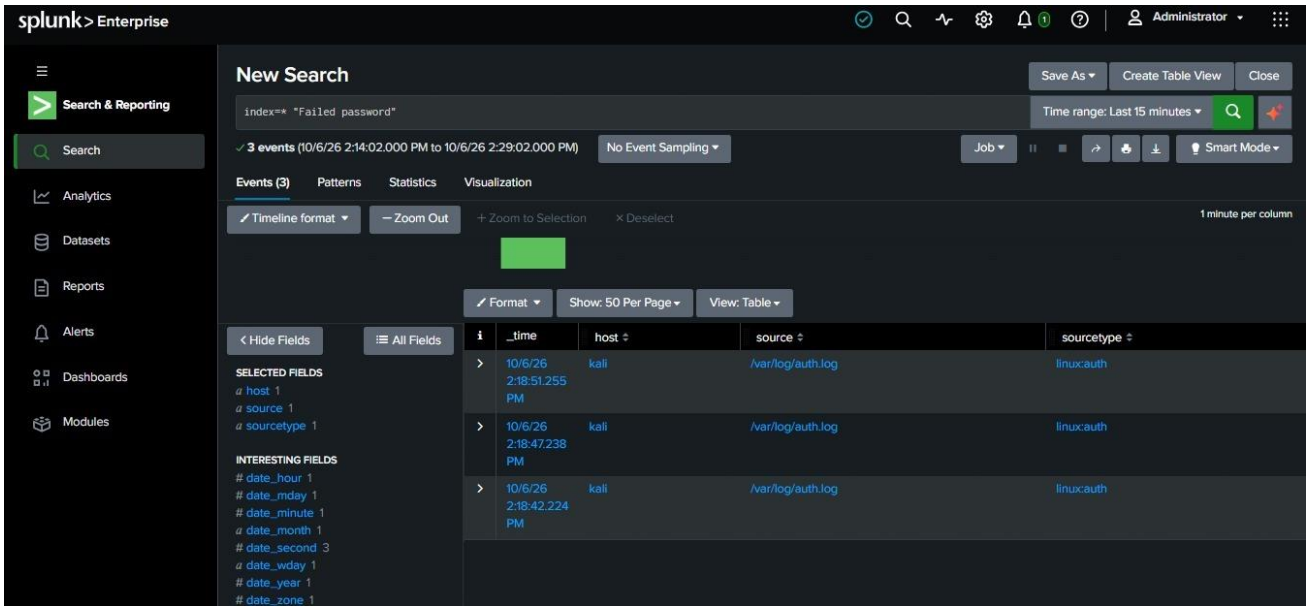


Figure 11. *Splunk search returning three Failed password events from host kali.*

What the screenshot shows

- **3 events** between 10/6/26 2:14:02 PM and 2:29:02 PM were returned.
- **Timeline:** a single green bar (1 minute per column) shows all three events fell within the same minute, around 2:18 PM.
- **Event times:** 2:18:42, 2:18:47 and 2:18:51 PM, about five seconds apart, which matches typing three wrong passwords.
- **Fields:** `host = kali`, `source = /var/log/auth.log`, `sourcetype = linux:auth`. This confirms that Splunk is receiving the same file that was inspected with grep.
- The `index=*` part searches every index. That is convenient in a small lab; in production a specific index should be named to keep searches fast.
- **The quoted phrase** `"Failed password"` is the exact text sshd writes for each wrong password (Section 4.8).

> **WHY ONLY THREE EVENTS?**
> The earlier rounds (at 13:55 and 14:06) happened before this 15-minute window began at 14:14, so they are not counted. Widening the time range (for example to Last 24 hours) shows them as well.

5.2 The Real-Time Alert

A real-time alert named `ssh brute force` was created in Splunk so that failed SSH logins are flagged as they arrive, without anyone running the search by hand. Its activity appears under **Alerts → Triggered Alerts**.

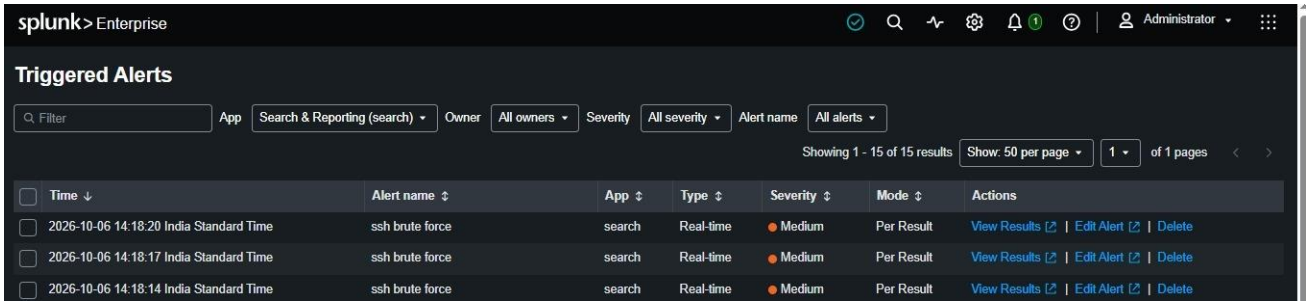


Figure 12. *Triggered Alerts page: repeated "ssh brute force" alerts of Medium severity.*

Setting / observation	Value in the screenshot
Alert name	`ssh brute force`
App	Search & Reporting (`search`)
Type	Real-time
Severity	Medium
Mode	Per Result
Visible trigger times	2026-10-06 14:18:14, 14:18:17 and 14:18:20 India Standard Time
Total triggered	15 (`Showing 1 - 15 of 15 results`)
Available actions	View Results, Edit Alert, Delete

What the settings mean

- **Real-time:** the search runs continuously, so the alert fires within seconds of the log line being indexed, instead of waiting for a schedule.
- **Per Result:** one alert is raised for every matching event. Nothing is missed, but a single login attempt produces several alerts, which is why 15 appear.
- **Medium severity:** a label for triage. Repeated failed logins deserve attention but are not yet a confirmed compromise.

> **VERIFICATION RESULT**
> The alert did not just exist; the Triggered Alerts page proves it fired in response to the SSH events generated in this lab.

5.3 Recommended Improvement: Rolling-Window Detection

One failed password is normal (a mistyped password). A brute-force attack is **several failures from the same source in a short time**. A stronger rule therefore counts failures per source address per minute. The search below is a recommended design for this lab (it was not part of the captured screenshots):

```
index=* sourcetype=linux:auth "Failed password"
| rex "from (?<src_ip>\d{1,3}(?:\.\d{1,3}){3}) port"
| bin _time span=1m
| stats count by _time, host, src_ip
| where count >= 3
```

- `rex` extracts the source address from the message text into a field called `src_ip`.

- `bin _time span=1m` groups events into one-minute buckets.
- `stats count by ...` counts failures per bucket, host and source address.
- `where count >= 3` keeps only sources with three or more failures in a minute. In this lab each round produces exactly three, so the threshold fits; on a real server it would be tuned higher.

Saved as an alert, this can run on a schedule (for example every minute over the last five minutes) and trigger when the number of results is greater than zero, with throttling per `src_ip` to avoid repeated notifications.

6. Challenges & Troubleshooting

This section documents the errors encountered during the lab and how each was diagnosed and resolved. They are typical of the first hours of working with SSH logs on Linux.

6.1 SSH Service Inactive (dead)

Problem: `sudo systemctl status ssh` reported `Active: inactive (dead)` (Figure 1). With the service stopped, nothing is listening on port 22, so SSH connections to the VM would be refused and no authentication events could be generated.

Root Cause: On Kali the SSH server is shipped disabled for security. The status line `disabled; preset: disabled` shows it is neither started at boot nor enabled by default.

Resolution: The service was started manually, then checked again:

Command used:
```
sudo systemctl start ssh
sudo systemctl status ssh
```

The status changed to `active (running)` and the log showed `Server listening on 0.0.0.0 port 22`. A manual `start` lasts only until the next reboot, so the command has to be repeated after restarting the VM. `sudo systemctl enable ssh` would make it start at boot, but for security it is better to leave it disabled and start it only when needed.

6.2 "No such file or directory" for pract_auth.log

Problem: `sudo grep "Failed Attempts" /var/log/pract_auth.log` returned `grep: /var/log/pract_auth.log: No such file or directory` (Figure 4).

Root Cause: The path was wrong. `pract_auth.log` is not a file on this system. Kali and other Debian-based systems store SSH and sudo authentication events in `/var/log/auth.log`.

Resolution: The search was pointed at the correct file. Listing the log directory first is a quick way to confirm the name:

Command used:
```
ls /var/log | grep auth
sudo grep -E "Failed password|Invalid user" /var/log/auth.log
```

If `auth.log` is ever missing (some minimal installs log only to the systemd journal), the same events can be read with `sudo journalctl -u ssh`.

6.3 Search Phrase Matched the Wrong Lines

Problem: After the path was fixed, `sudo grep "Failed Attempts" /var/log/auth.log` returned two lines, but they were `sudo` records of the earlier grep commands, not SSH failures (Figure 5).

Root Cause: "Failed Attempts" is not a phrase OpenSSH writes. `grep` is also case-sensitive, so even a similar phrase in different case would not match. The only lines containing the text were the entries sudo wrote about the commands that contained it.

Resolution: The search was changed to the phrases sshd actually writes (their exact wording is visible in the `tail` output in Figure 9), joined with an OR:

Command used: `sudo grep -E "Failed password|Invalid user" /var/log/auth.log`

> **LESSON**
> Look at a few real log lines before writing a search. A detection can only match the text that the log really contains. The same lesson applies to Splunk: the search `index=* "Failed password"` works because that is the exact message in the log.

6.4 Extra sudo Lines Appearing in the Results

Problem: The filtered output (Figure 6 and Figure 8) contained `sudo` lines such as the one at 14:05:25 that are not SSH events.

Root Cause: `sudo` logs every command it runs, including the full text of the grep command. That text contains `Failed password|Invalid user`, so the log line of the search matches the search itself. Each new search therefore adds one more matching line.

Resolution: The sudo lines can be removed by filtering them out with a second grep (this second grep does not use sudo, so it adds no new line):

Command used: `sudo grep -E "Failed password|Invalid user" /var/log/auth.log | grep -v "sudo:"`

6.5 Summary of Errors

#	Error / symptom	Cause	Fix	Evidence
1	SSH service `inactive (dead)`	Service disabled by default on Kali	`sudo systemctl start ssh`	Figure 1
2	`No such file or directory`	Wrong log file name (`pract_auth.log`)	Use `/var/log/auth.log`	Figure 4
3	Only sudo lines matched	"Failed Attempts" is not an SSH message	Search `Failed password|Invalid user`	Figure 5
4	Extra sudo lines in results	sudo logs the grep command itself	Pipe through `grep -v "sudo:"`	Figures 6, 8

6.6 Messages That Look Like Errors but Are Normal

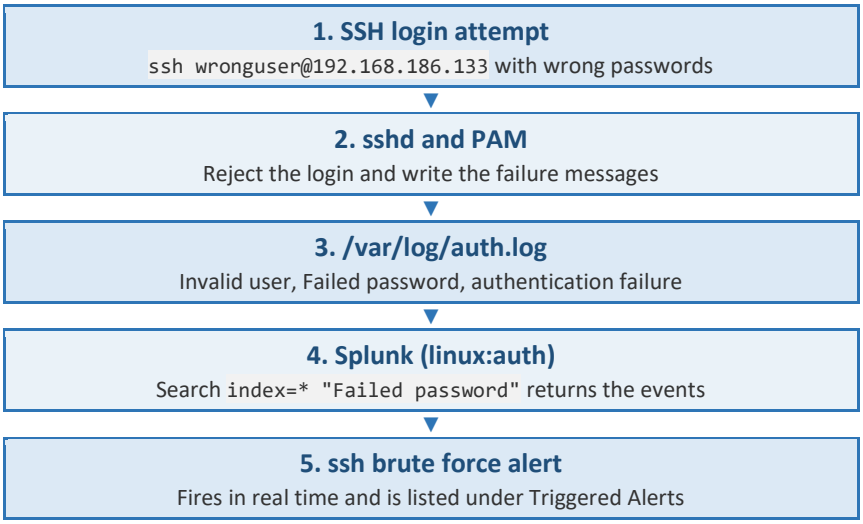
Message	Why it appears
`The authenticity of host ... can't be established`	First connection to a server whose key is not in `known_hosts` yet. It does not appear on later connections (Figure 7).
`Permission denied, please try again.`	A wrong password. This is the expected result of the test.
`Permission denied (publickey,password).`	All allowed methods failed and the client stopped after three tries.
`[sudo] password for kali:`	sudo asks for the user's own password the first time it is used in a session.

7. Verification & Testing

7.1 Test Results

Test	Expected result	Observed evidence	Result
SSH service status	Active / running	`active (running)`, PID 16084 (Figure 1)	✓ Pass
Listening port	Port 22 open	`0.0.0.0` and `::` port 22 (Figure 1)	✓ Pass
Server address	IP identified	`192.168.186.133/24` on `eth0` (Figure 2)	✓ Pass
Failed login generated	Login rejected	`Permission denied (publickey,password)` (Figures 3, 7, 10)	✓ Pass
Server-side evidence	Failure messages in log	Invalid user, Failed password, PAM failure (Figures 6, 8, 9)	✓ Pass
Log filtering	Only SSH failures shown	`grep -E` output (Figure 6)	✓ Pass
Splunk ingestion	Events searchable	3 events, `host=kali`, `linux:auth` (Figure 11)	✓ Pass
Real-time alert	Alert fires	`ssh brute force`, 15 triggered (Figure 12)	✓ Pass

7.2 End-to-End Evidence Chain



PIPELINE VALIDATION: COMPLETE
Kali SSH attempt → auth.log → Splunk → real-time alert. The same failed logins are visible at every stage, from the raw log to the SIEM alert.

8. Security Analysis

8.1 Indicators Observed

- Repeated `Failed password` messages, three per connection and several connections in a short period
- `Invalid user` messages, which suggest username guessing
- PAM `authentication failure` lines with the same `rhost`

- A single source address (`192.168.186.133`) behind all the failures
- Regular spacing between attempts (about 5 to 9 seconds here because they were typed by hand; automated tools would be much faster)
- Multiple real-time alerts on the Triggered Alerts page

8.2 MITRE ATT&CK Mapping

This behavior corresponds to **T1110.001: Brute Force: Password Guessing** in the MITRE ATT&CK framework (tactic: Credential Access).

8.3 Recommended Response Workflow

Step	Action
1	Validate the alert: confirm source address, target host and time range in the raw events.
2	Review which usernames were tried (real accounts, or invalid names such as `wronguser`).
3	Build the timeline of failures and check how many occurred per minute.
4	Search for a `Accepted password` or `Accepted publickey` event from the same source after the failures, which would indicate a successful compromise.
5	Correlate with other host and network evidence.
6	Contain according to policy (for example block the source address at the firewall) and document the incident.

8.4 Hardening the SSH Server

Detection should be paired with prevention. Typical measures in `/etc/ssh/sshd_config` are:

```
PasswordAuthentication no
PermitRootLogin no
MaxAuthTries 3
```

- After editing, apply the change with `sudo systemctl restart ssh`.
- **Set up and test key-based login first.** Turning off password authentication without a working key will lock you out.
- Install a ban tool such as `fail2ban` (`sudo apt install fail2ban`) to block sources automatically after repeated failures.
- Restrict port 22 with a firewall and stop the service when it is not needed: `sudo systemctl stop ssh`.

9. Limitations & Future Enhancements

9.1 Limitations

- The simulation is small (three attempts per connection, a few connections), and the source and target are the same VM.
- The alert is **Per Result**, so every matching event triggers a separate alert (15 in total here). This is noisy and does not measure how many failures happened within a time window.

- The search uses `index=*` and a text match. The Splunk field list in the screenshot shows `host`, `source`, `sourcetype` and date fields only, so the source address and username have to be extracted with `rex` before they can be used in statistics.
- Only failed logins are covered; successful logins after the failures are not yet correlated.

9.2 Future Enhancements

- Replace the Per Result alert with the rolling-window rule from Section 5.3
- Extract `src_ip` and `user` as fields and build a dashboard (failures over time, top source addresses, targeted usernames)
- Add a correlation search for a successful login that follows repeated failures
- Send the logs from a separate attacker VM so the source address is a genuine remote host
- Repeat the same scenario with an ELK stack SIEM and compare how detection works in both tools

10. Conclusion

This project showed, step by step, how a failed SSH login becomes a SIEM alert. The OpenSSH server was started and verified, failed logins were generated with an invalid username, and the resulting `Invalid user`, `Failed password` and PAM messages were found in `/var/log/auth.log`, interpreted line by line, and then confirmed in Splunk with `host=kali` and `sourcetype=linux:auth`. The real-time `ssh brute force` alert fired repeatedly, proving the detection works from end to end.

The troubleshooting was just as valuable as the success path. A stopped service, a wrong file name and a search phrase that did not exist in the log each looked like a failure of the lab, but each had a simple cause. The shared lesson is that a detection can only be as good as its match with the real log source and message format.

The next step is to move from per-event alerts to a rolling-window rule that counts failures per source, and to extend the lab with a separate attacker machine and a successful-login correlation.

> **FINAL OUTCOME**
> The lab provides a practical foundation for SOC exercises in authentication monitoring, brute-force detection, alert tuning and incident response.

11. References

1. OpenSSH manual pages: https://www.openssh.com/manual.html
2. Splunk Enterprise Search Manual: https://docs.splunk.com/Documentation/Splunk/latest/Search
3. Splunk Enterprise Alerting Manual: https://docs.splunk.com/Documentation/Splunk/latest/Alert
4. MITRE ATT&CK, T1110.001 Password Guessing: https://attack.mitre.org/techniques/T1110/001/
5. Kali Linux documentation: https://www.kali.org/docs/

— End of Report —